

MIDI Maze – MIDI over SIO (POKEY Serial) Handler Analysis

Expanded edition: annotated assembly excerpts, baud-rate math verification, and C-like pseudocode.

Executive Summary

The ROM does **not** call the Atari OS SIO/CIO transfer routines for MIDI traffic. Instead it uses **POKEY's serial hardware directly** via **SERIN/SEROUT** (\$D20D). It still hooks the OS interrupt vectors **VSERIN** and **VSEROR** so that POKEY serial interrupts dispatch into the game's code. Receive and transmit are handled using **256-byte ring buffers** with interrupt-driven fill/drain. No clear evidence of software handshaking via SIO **PROCEED/CMD** lines (no PORTA/PORTB polling) was found; the design relies on POKEY serial IRQs and simple buffer backpressure.

Annotated Assembly Excerpts

The following excerpts are taken from the disassembly and annotated to explain what each routine is doing.

Init (hooks VSERIN/VSEROR + config POKEY)

```
LBEFD    SEI                                ; disable IRQs while patching vectors / HW
          LDA #$3C
          STA PACTL                          ; PIA control: put SIO-related lines in known mode
          LDA #$3C
          STA PBCTL

          LDY #$03
LBF0A    LDA VSERIN,Y                        ; save OS serial vector bytes...
          STA L3DF8,Y                        ; ...into backup storage
          DEY
          BPL LBF0A

          LDA #$9C                          ; VSERIN = $BE9C (our serial input ISR)
          STA VSERIN
          LDA #$BE
          STA VSERIN+1

          LDA #$AC                          ; VSEROR = $BEAC (our serial output ISR)
          STA VSEROR
          LDA #$BE
          STA VSEROR+1

          LDA #$00
          LDY #$05
LBF2B    STA L0082,Y                        ; clear RX/TX indices + flags (L0082..L0087)
          DEY
          BPL LBF2B

          LDA #$28
          STA AUDCTL                          ; join ch3+ch4; clock ch3 at ~1.79MHz
          LDA #$A0
          STA AUDC3                          ; "pure tone" distortion + volume bits (used for clocking)
          STA AUDC4
          LDA #$15
          STA AUDF3                          ; divisor low byte
          LDA #$00
          STA AUDF4                          ; divisor high byte

          LDA #$13
          STA SSKCTL                          ; shadow copy
          STA SKCTL                          ; serial control (sets serial mode bits)
          STA SKREST                         ; clear serial status latches
```

```

LDA POKMSK
AND #$F0
ORA #$30          ; enable POKEY serial IRQ bits in mask
STA POKMSK
STA IRQEN

CLI              ; IRQs back on
LDA #$34
STA PACTL
LDY #$00
RTS

```

Serial input ISR (VSERIN -> SERIN -> RX ring)

```

; VSERIN points to $BE9C, which starts with a one-byte prologue: TYA
.byte $98          ; TYA (save Y in A)
LBE9D  PHA          ; push saved Y
        LDA SERIN    ; read received byte from POKEY
        LDY L0082     ; Y = RX write index
        STA L2D00,Y   ; rx_buf[rx_w] = byte
        INC L0082     ; rx_w++ (wraps naturally at 256)
        PLA          ; restore saved Y into A
        TAY          ; restore Y
        PLA          ; restore A saved by interrupt entry
        RTI          ; return from interrupt

```

Serial output ISR (VSEROR -> TX ring -> SEROUT)

```

; VSEROR points to $BEAC, which also starts with TYA
.byte $98          ; TYA (save Y in A)
LBEAD  PHA          ; push saved Y
        LDY L0084     ; Y = TX read index
        CPY L0085     ; read == write ?
        BEQ LBEBF     ; yes -> no queued bytes

        LDA L2E00,Y   ; A = tx_buf[tx_r]
        STA SEROUT    ; transmit next byte
        INC L0084     ; tx_r++
        JMP LBEC3

LBEBF  LDA #$00
        STA L0086     ; tx_active = 0 (idle)

LBEC3  PLA
        TAY          ; restore Y
        PLA          ; restore A
        RTI

```

Send-byte API (kick TX or enqueue)

```

LBEC7  SEI          ; protect queue state vs ISR
        LDY L0086
        BNE LBED5   ; if TX active, enqueue

        STA SEROUT   ; idle: write directly to start TX immediately
        INC L0086    ; tx_active = 1
        CLI
        LDY #$00
        RTS

LBED5  LDY L0085     ; Y = TX write index
        STA L2E00,Y  ; tx_buf[tx_w] = byte
        INY
        STY L0085    ; tx_w++
        CLI

; backpressure: if queue nearly full, busy-wait until ISR drains it
LBEDE  TYA
        SEC
        SBC L0084    ; (tx_w - tx_r)
        CMP #$F8

```

```
BCS LBEDE
LDY #$00
RTS
```

Receive-byte API (blocking dequeue)

```
LBEE9  LDY L0083          ; Y = RX read index
        CPY L0082          ; rx_r == rx_w ?
        BEQ LBEE9          ; yes -> spin until a byte arrives

        LDA L2D00,Y        ; A = rx_buf[rx_r]
        INC L0083          ; rx_r++
        LDY #$00
        RTS
```

Baud Rate Verification

The ROM programs POKEY as follows in the init routine: **AUDCTL = \$28** (join channels 3+4 into 16-bit, clock channel 3 from ~1.79 MHz) **AUDF3 = \$15**, **AUDF4 = \$00** (16-bit divisor value $N = \$0015 = 21$) **SKCTL = \$13** (serial mode bits set; also enables keyboard debounce/scan bits) A commonly used serial bit-rate relationship for POKEY when using the joined 16-bit timer is:

$$\text{baud} \approx \text{clock} / (2 \cdot (N + 7))$$

This is consistent with reverse-calculations people do for 19.2k SIO (NTSC divisor \$28 → ~19,040 baud). Using that same relationship here with $N=21$ yields: NTSC (1,789,772.5 Hz): **31,960.2 baud** PAL (1,773,447.5 Hz): **31,668.7 baud** Interpretation: This is in the neighborhood of the MIDI standard rate (31,250 baud), but is not an exact match. It may indicate (a) a slightly different mode/clock source than assumed, (b) a “good enough” timing choice, or (c) a protocol that is MIDI-like but not strictly MIDI-spec timing. References used for the calculation guidance: AtariAge discussion: divisor/ baud back-calculation and offset usage for 19.2k SIO. AtariAge discussion: timer fires every $N+7$ cycles and effective rate is halved for serial.

C-like Pseudocode

This pseudocode matches the observed structure: vector hooking, ISR-driven ring buffers, and simple send/receive APIs.

```
/*
 * MIDI Maze: custom POKEY-serial (SIO) driver sketch
 * (based on disassembly of the 128KB cartridge ROM)
 */

// Ring buffers (256 bytes each, indices wrap naturally)
uint8_t rx_buf[256]; // corresponds to L2D00
uint8_t tx_buf[256]; // corresponds to L2E00

volatile uint8_t rx_w = 0; // L0082: RX write index (ISR increments)
volatile uint8_t rx_r = 0; // L0083: RX read index (main code increments)

volatile uint8_t tx_r = 0; // L0084: TX read index (ISR increments)
volatile uint8_t tx_w = 0; // L0085: TX write index (main code increments)

volatile uint8_t tx_active = 0; // L0086: 0=idle, nonzero=TX in progress

void midi_init(void) {
    disable_irq();

    // Put PIA/SIO control into a known state (does not appear to do PROCEED handshaking)
    PACTL = 0x3C;
    PBCTL = 0x3C;

    // Save and replace OS serial vectors (VSERIN/VSEROR)
    saved_vserin = VSERIN;
    saved_vseror = VSEROR;
    VSERIN = isr_serial_in;
    VSEROR = isr_serial_out;

    // Clear indices/flags
    rx_w = rx_r = 0;
    tx_w = tx_r = 0;
    tx_active = 0;

    // Program POKEY for serial timing
    AUDCTL = 0x28; // join ch3+ch4, clock ch3 at ~1.79MHz
    AUDC3 = 0xA0; // tone/noise control values used with serial clocking
    AUDC4 = 0xA0;
    AUDF3 = 0x15; // low byte of divisor
    AUDF4 = 0x00; // high byte of divisor
    SKCTL = 0x13; // serial/keyboard control (serial mode bits set here)
```

```

    SKREST = 0x00;        // reset serial status latches

    // Enable POKEY serial IRQs via OS shadow + hardware register
    POKMSK = (POKMSK & 0xF0) | 0x30;
    IRQEN = POKMSK;

    enable_irq();

    PACTL = 0x34; // post-init tweak
}

// ISR: serial byte received
void isr_serial_in(void) {
    uint8_t b = SERIN;        // read received byte
    rx_buf[rx_w++] = b;        // store and advance write index
}

// ISR: serial output needs a byte (or output done)
void isr_serial_out(void) {
    if (tx_r == tx_w) {
        tx_active = 0;        // queue empty
        return;
    }
    SEROUT = tx_buf[tx_r++];    // write next byte to transmit
}

// Send a byte (non-OS SIO path)
void midi_send(uint8_t b) {
    disable_irq();            // protect shared indices vs ISR

    if (!tx_active) {
        SEROUT = b;            // kick transmit immediately
        tx_active = 1;
        enable_irq();
        return;
    }

    // Otherwise enqueue
    tx_buf[tx_w++] = b;
    enable_irq();

    // Crude backpressure: wait if nearly full
    while ((uint8_t)(tx_w - tx_r) >= 0xF8) {
        // spin until ISR drains the buffer
    }
}

// Blocking receive
uint8_t midi_recv_blocking(void) {
    while (rx_r == rx_w) {
        // spin until ISR receives something
    }
    return rx_buf[rx_r++];
}

```